

Practical Week-8

8a)

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a, *b;

    a = (int *)malloc(5 * sizeof(int));

    b = (int *)calloc(5, sizeof(int));

    a = (int *)realloc(a, 10 * sizeof(int));

    free(a);
    free(b);

    return 0;
}
```

```
mahij@Mahija:~$ nano valgrind.c
mahij@Mahija:~$ gcc valgrind.c -o valgrind
mahij@Mahija:~$ valgrind --leak-check=full ./valgrind
==2327== Memcheck, a memory error detector
==2327== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2327== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2327== Command: ./valgrind
==2327==
==2327==
==2327== HEAP SUMMARY:
==2327==     in use at exit: 0 bytes in 0 blocks
==2327==   total heap usage: 3 allocs, 3 frees, 80 bytes allocated
==2327==
==2327== All heap blocks were freed -- no leaks are possible
==2327==
==2327== For lists of detected and suppressed errors, rerun with: -s
==2327== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
mahij@Mahija:~$ |
```

8b)

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main()
{
    int x = 100;

    if (fork() == 0)
    {
        printf("Child Before=%d\n", x);

        x = 500;

        printf("Child After=%d\n", x);
    }
    else
    {
        sleep(2);

        printf("Parent=%d\n", x);
    }

    return 0;
}
```

```
mahij@Mahija: ~
mahij@Mahija:~$ nano leaks.c
mahij@Mahija:~$ gcc leaks.c -o leaks
mahij@Mahija:~$ ./leaks
Child Before=100
Child After=500
Parent=100
mahij@Mahija:~$ |
```